



C#.NET

Mr. Bangar Raju

 **NARESHO**
technologies

Different kinds of Variables in a class

Understanding Different Types of Variables in C# Classes

By Albert Harmon

Published October 22, 2016

🕒 4 min read

[Education](#)



TLDR This blog post explores the four main types of variables in C# classes: non-static, static, constant, and read-only variables. It explains their definitions, behaviors, initialization processes, and differences, providing a comprehensive guide for

developers to choose the appropriate variable type based on their needs.

In this blog post, we will explore the different kinds of variables that can be declared within a class in C#. Understanding these variable types is crucial for effective programming and memory management. We will cover four main types of variables: non-static variables, static variables, constant variables, and read-only variables.

What is a Variable?

A variable is a storage location identified by a name that can hold a value. In the context of a class, variables are used to store data that can be manipulated by the methods of the class.

Types of Variables in C# Classes

1. Non-Static Variables

Non-static variables, also known as instance variables, are declared without the static modifier. Each instance of a class has its own copy of non-static variables. This means that if you create multiple instances of a class, each instance will have its own separate copy of the non-static variables.

Behavior of Non-Static Variables

- **Initialization:** Non-static variables are initialized only when an instance of the class is created. If no instances are created, the memory for these variables is not allocated.
- **Access:** To access a non-static variable, you must create an instance of the class. For example:

```
Program p1 = new Program();  
Console.WriteLine(p1.X);
```

2. Static Variables

Static variables are declared using the static modifier. Unlike non-static variables, static variables are shared among all instances of a class. There is only one copy of a static variable, regardless of how many instances of the class are created.

Behavior of Static Variables

- **Initialization:** Static variables are initialized once when the class is loaded, before any instances are created.
- **Access:** Static variables can be accessed directly using the class name without creating an instance. For example:

```
Console.WriteLine(Program.Y);
```

3. Constant Variables

Constant variables are declared using the `const` keyword. These variables must be initialized at the time of declaration and cannot be modified afterward.

Behavior of Constant Variables

- **Initialization:** Constant variables must be assigned a value at the time of declaration. For example:

```
const float PI = 3.14f;
```

- **Access:** Like static variables, constant variables are shared across all instances of the class and do not require an instance to access.

4. Read-Only Variables

Read-only variables are declared using the `readonly` keyword. Unlike constant variables, read-only variables can be initialized either at the time of declaration or within a constructor.

Behavior of Read-Only Variables

- **Initialization:** Read-only variables can be assigned a value at declaration or within a constructor. For example:

```

public readonly bool Flag;
public Program() {
    Flag = true;
}

```

- **Access:** Each instance of the class has its own copy of read-only variables, similar to non-static variables. However, once initialized, their values cannot be changed.

Key Differences Between Variable Types

Variable Type	Initialization Timing	Modifiability	Copies per Instance
Non-Static	After instance creation	Yes	One per instance
Static	Once when the class is loaded	Yes	One for the class
Constant	At declaration only	No	One for the class
Read-Only	At declaration or in constructor	No (after initialization)	One per instance

Conclusion

In summary, understanding the different types of variables in C# classes is essential for effective programming. Each variable type serves a specific purpose and has its own behavior regarding initialization, access, and modifiability. When developing applications, it is important to analyze the requirements and choose the appropriate variable type

accordingly. By doing so, you can optimize memory usage and ensure that your code behaves as expected.

Thank you for reading!